Fine-Tuning — Complete Guide

Section 15 | When RAG Is Not Enough

Author: Suresh D R | AI Product Developer & Technology Mentor

---

Table of Contents

1. What Is Fine-Tuning — In Plain English
2. Why Fine-Tuning Actually Exists — The Real Motivation
3. Why RAG Alone Is Sometimes Not Enough
4. RAG vs Fine-Tuning — When to Use Which
5. What Happens Inside the Model During Fine-Tuning
6. Types of Fine-Tuning — Full Taxonomy
7. Self-Supervised Fine-Tuning — No Labelled Data Needed
8. Fine-Tuning on the TechNova HR Document
9. PEFT — Parameter-Efficient Fine-Tuning
10. LoRA — How It Actually Works (With Full Calculations)
11. QLoRA — Fine-Tuning on a Laptop
12. Training Parameters — What They Mean and How to Set Them
13. What Gets Updated — A Layer-by-Layer View
14. Common Problems and Fixes
15. Summary — When to Do What

---

1. What Is Fine-Tuning — In Plain English

Fine-tuning means taking a pre-trained LLM and continuing to train it on your own specific data — so it becomes an expert in your domain, speaks your company's language, and answers in exactly the style and format you need.

The Best Analogy

Think of a medical student who just graduated from general medical school. They know medicine broadly — anatomy, physiology, pharmacology, general treatment. But now they join a hospital that specialises in cardiology. For the first few months, the senior cardiologists teach them specifically about heart conditions, the hospital's protocols, the specific terminology used, the types of reports the hospital writes.

The student does not forget general medicine. They just become much sharper and more specialised in cardiology.

That is fine-tuning. The LLM already knows language, reasoning, and general knowledge. You teach it specifically about your domain — your documents, your terminology, your expected output format, your company's way of responding.

What Fine-Tuning Changes

```
Before fine-tuning:
  You ask: "What is the leave encashment policy?"
  LLM: "Leave encashment policies vary by company and country.
         Generally, employees can convert unused leave into monetary
         compensation based on their daily salary rate..."
  → Generic answer. Does not know YOUR company's policy.

After fine-tuning on TechNova HR document:
  You ask: "What is the leave encashment policy?"
  LLM: "At TechNova, employees can encash up to 30 days of earned leave
         per year. The encashment rate is calculated as basic salary / 26
         working days. Applications must be submitted through HRConnect
         by the 10th of each month."
  → Specific, accurate, company-aligned answer.
```

---

2. Why Fine-Tuning Actually Exists — The Real Motivation

This is the section most guides skip. Before diving into methods, you need to understand **why** fine-tuning was invented and what fundamental problem it solves.

The Core Problem with Pre-trained Models

LLMs are pre-trained on the internet — Wikipedia, books, code, news, forums, research papers. This gives them remarkable general ability. But general ability has a critical gap:

The model knows nothing about YOU.

It does not know your company's policies, terminology, or processes. It does not know your expected output format. It does not know what to say and what NOT to say. It does not know the implicit reasoning that connects your domain knowledge together.

Every time you query it, you have to explain context. Every answer is generic. Every response style must be steered by prompts. This is fine for general tasks — but breaks down for production applications where users expect accurate domain-specific answers, responses must follow a specific format, and the model must refuse gracefully when it does not know something.

Why Not Just Prompt It Better?

Prompt engineering is powerful but has hard limits:

Limitation	What Happens
Context window limit	You can only fit so much into one prompt
Repetition cost	You pay token cost on every single query
Style consistency	Prompt changes slightly → output style drifts
Implicit knowledge	Some domain logic cannot be expressed as instructions
Latency	Longer prompts = slower responses

Fine-tuning eliminates these limits by making the knowledge part of the model itself.

> Fine-tuning exists because pre-trained models are general-purpose, but real production applications need a specialist. Fine-tuning converts a generalist into a specialist for your domain.

---

3. Why RAG Alone Is Sometimes Not Enough

RAG is powerful. You give the model relevant chunks and it answers from them. But RAG has real limitations — and understanding them shows exactly when fine-tuning is needed.

RAG Problem 1 — Chunk Loss

Even with good retrieval, some information gets missed.

```
Your document has this content:

  Section 3.2:  "Maternity leave is 26 weeks, fully paid."
  Section 7.1:  "Leave encashment applies to earned leave only,
                 not maternity or paternity leave."
  Section 9.4:  "Salary during maternity leave is paid as per
                 the standard payroll cycle, not in advance."

User asks: "Can I get maternity leave salary in advance?"

Retriever fetches: Section 3.2 (high similarity to "maternity leave")
                   Section 9.4 (partial match)
                   MISSES Section 7.1 (different wording, low similarity)

LLM answers from only two of the three relevant sections.
The answer might be incomplete or slightly wrong.
```

Simple analogy: You ask a librarian for books about maternity leave. They bring you 3 books. But the most relevant paragraph is in a finance book in a different section — the librarian never thought to look there.

RAG Problem 2 — Style Mismatch

The model knows the facts but does not sound like your company.

```
TechNova needs answers in this format:
  "As per TechNova HR Policy Section X.X, [policy]. For queries,
   contact hr@technova.com or raise a ticket on HRConnect."

RAG gives: "The maternity leave policy provides 26 weeks of paid leave."
            → Correct fact. Wrong tone, format, and missing contact info.
```

No matter how good your RAG retrieval is, the LLM does not know your company's response style without being taught it.

RAG Problem 3 — Hallucination Under Uncertainty

When retrieved chunks are slightly irrelevant, the LLM fills the gaps with plausible-sounding but incorrect information.

```
User asks about a specific edge case not clearly covered in any chunk.
Retriever finds weakly related chunks.
LLM: "Based on the policy, I believe employees can..."
         ↑ "I believe" = hallucination territory

A fine-tuned model trained on your documents learns to say:
"This specific case is not covered in the current policy.
 Please contact HR for clarification." → much safer response
```

RAG Problem 4 — Implicit Knowledge

Some knowledge cannot be expressed as chunks. It is the background understanding that connects all the chunks together.

```
Chunk A: "L3 employees receive Rs 1.4L–2.1L salary"
Chunk B: "Performance review determines annual increment"
Chunk C: "ESOP vesting starts after 2 years"

User asks: "I am an L3 with 18 months of service, good performance.
            What should I expect in my next review?"

This requires combining A + B + C with understanding of the company's
progression logic — not just retrieving individual chunks.
A fine-tuned model builds this holistic understanding.
```

RAG Problem 5 — Response Consistency

Different retrieval runs return slightly different chunks → slightly different answers → inconsistency that erodes user trust.

A fine-tuned model always gives the same answer because the knowledge is baked into the weights, not retrieved from external chunks.

---

4. RAG vs Fine-Tuning — When to Use Which

Use RAG when:

- Documents change frequently (new policies, new versions)

- You need to cite sources ("as per section 3.2")
- You need transparency in what was retrieved
- You cannot retrain every time the document updates
- Quick setup is a priority

Use Fine-Tuning when:

- You need a specific tone and response style
- The model needs to know your company's terminology precisely
- You need consistent answers regardless of retrieval quality
- The task requires reasoning across multiple sections
- You need the model to say "I don't know" for out-of-scope questions
- Latency matters (no retrieval step needed at inference time)

Use BOTH together (the best production setup):
Fine-tune for style, tone, and domain understanding. Use RAG for current, citable, document-specific facts. The fine-tuned model understands the domain better AND the RAG retrieves the specific facts.

---

5. What Happens Inside the Model During Fine-Tuning

The LLM as a Giant Function

An LLM is a neural network with billions of numbers called weights or parameters. When you give it input text, those numbers together compute the output text. Pre-training learned general language understanding and world knowledge by adjusting those numbers over trillions of tokens.

Fine-tuning continues adjusting those same numbers — but now with your specific data.

Do ALL Weights Get Updated?

Full Fine-Tuning updates all weights. All billions of parameters adjusted on your data. Expensive. Can cause catastrophic forgetting — the model forgets general knowledge to learn your specific data. Requires a large GPU cluster.

Partial Fine-Tuning freezes early layers and updates only the last few layers. Early layers contain general language understanding — keep those. Last layers contain task-specific behaviour — update those. Cheaper than full fine-tuning, less forgetting.

LoRA is the most practical approach. No existing weights are updated. Small new matrices called adapters are added alongside existing weights. Only the adapters are trained. 99% cheaper than full fine-tuning. Same model quality. Covered in full detail in Sections 9 and 10.

Why Not Always Full Fine-Tuning?

```
Full fine-tuning on LLaMA 3 70B:
  Weights:    70B × 4 bytes (float32) = 280 GB
  Gradients:  70B × 4 bytes           = 280 GB
  Optimizer:  70B × 8 bytes           = 560 GB  (Adam: 2 moments × 4 bytes)
  TOTAL:                               ~840 GB GPU memory needed

  Top-of-the-line GPU (A100): 80 GB memory.
  You need 10-11 A100s just to load the model for training.
  Each A100 costs ~$30,000.

LoRA fine-tuning on LLaMA 3 70B:
  Add small adapter matrices → only a few hundred MB to train.
  Can run on a single 24 GB consumer GPU.
  Training cost: a few dollars on a cloud instance.
```

---

6. Types of Fine-Tuning — Full Taxonomy

```
Fine-Tuning
  │
  ├── Supervised Fine-Tuning (SFT)
  │     Labelled data needed: YES — (input, output) pairs
  │     Example: Question → Correct Answer pairs
  │
  ├── Self-Supervised Fine-Tuning
  │     Labelled data needed: NO — raw text only
  │     The model predicts the next word. The document is its own label.
  │
  ├── Instruction Fine-Tuning
  │     Labelled data needed: YES — (instruction, input, output) triples
  │     Example: "Answer HR questions formally" → examples
```

```
   │
   ├── RLHF — Reinforcement Learning from Human Feedback
   │     Labelled data needed: YES — human preference rankings at scale
   │     Used by OpenAI (ChatGPT), Anthropic (Claude), Google (Gemini)
   │     Cost: millions of dollars. Not practical for most companies.
   │
   └── PEFT — Parameter-Efficient Fine-Tuning  ← a METHOD, not a data type
         LoRA   — Low-Rank Adaptation        (covered in Section 10)
         QLoRA  — 4-bit quantised + LoRA     (covered in Section 11)
         Prefix Tuning, Prompt Tuning        (other lightweight methods)
```

> **Important:** LoRA and QLoRA are training *methods*, not data types. You can use LoRA with self-supervised data (no labels needed) or with supervised data. The method and the data type are independent choices.

Type 1 — Supervised Fine-Tuning (SFT)

Labelled data needed: YES

What it needs: Question → Answer pairs. You provide both the input and the correct output.

```
# Training data format
{"input":  "What is the maternity leave policy at TechNova?",
 "output": "TechNova provides 26 weeks of fully paid maternity leave.
            Employees with at least 80 days of service are eligible.
            Applications must be submitted 30 days in advance through
            HRConnect. Contact hr@technova.com for queries."}
```

You can auto-generate labels using GPT instead of writing them by hand:

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

llm = ChatOpenAI(model="gpt-4o", temperature=0.3)

qa_prompt = ChatPromptTemplate.from_messages([
    ("system",
     "Read this document section and generate 5 question-answer pairs. "
     "Questions should be things an employee would actually ask. "
```

```
     "Answers should be specific, complete, and based ONLY on the text. "
     "Return as JSON array: [{\"question\": ..., \"answer\": ...}]"),
    ("human", "{chunk}")
])

# 100 chunks × 5 pairs each = 500 training examples
# Generated in minutes, costs ~$2 in API fees
```

Type 2 — Self-Supervised Fine-Tuning

Labelled data needed: NO

What it needs: Raw text only. No labels, no question-answer pairs. Covered in full detail in Section 7.

Type 3 — Instruction Fine-Tuning

Labelled data needed: YES — (instruction, input, output) triples

```
{"instruction": "You are an HR assistant for TechNova. Answer questions
                 formally. Always cite the HR policy section. Always
                 provide contact information at the end.",
 "input":       "How much notice is required for planned leave?",
 "output":      "As per TechNova HR Policy Section 5.3, planned leave
                 requires a minimum of 7 days advance notice.
                 Contact hr@technova.com or raise a ticket on HRConnect."}
```

Type 4 — RLHF

Labelled data needed: YES — human preference rankings

```
Step 1: Fine-tune model on demonstrations (SFT)
Step 2: Human raters rank model outputs: Answer B > Answer A > Answer C
Step 3: Train a Reward Model to predict human rankings
Step 4: Use Reinforcement Learning to make the LLM score higher

This is expensive, complex, and needs human annotation at scale.
Not practical for most companies.
```

7. Self-Supervised Fine-Tuning — No Labelled Data Needed

The Core Idea — Next Word Prediction

In self-supervised learning, the model learns by **predicting the next word** at every position in the text. The document itself is the training signal — no human labelling is needed at all.

This is the same mechanism used during pre-training on the internet. You are simply continuing that process, but now with your own documents instead of the internet.

How it works, step by step:

The model reads text left to right. At each position, it sees all the words so far and must predict the next word. The actual next word in the document is the correct answer. The model adjusts its weights to get better at this prediction.

```
Your document text:
  "Maternity leave at TechNova is 26 weeks, fully paid."

Training examples created automatically:

  Input seen so far                          →   Next word to predict
  ──────────────────────────────────────────────────────────────────
  "Maternity"                                →   "leave"
  "Maternity leave"                          →   "at"
  "Maternity leave at"                       →   "TechNova"
  "Maternity leave at TechNova"              →   "is"
  "Maternity leave at TechNova is"           →   "26"
  "Maternity leave at TechNova is 26"        →   "weeks"
  "Maternity leave at TechNova is 26 weeks,"  →  "fully"
  "Maternity leave at TechNova is 26 weeks, fully"  →  "paid"

No human wrote these labels.
The document created them automatically.
One sentence = 8 training examples.
A 25,000-word document = ~25,000 training examples.
```

The Technical Mechanism — Cross-Entropy Loss

During training, the model outputs a probability distribution over its entire vocabulary (50,000+ words) for the next position. The loss function measures how wrong that prediction was. Backpropagation adjusts the weights to reduce this loss.

```
Input context: "Maternity leave at TechNova is"

Model outputs a probability over every word in vocabulary:
  "26"       → 0.42  ← correct answer
  "paid"     → 0.18
  "provided" → 0.12
  "given"    → 0.09
  ... 50,000 others ...

Loss = -log(0.42) = 0.38

After training on thousands of such examples:
  "26"   → 0.89  ← model now very confident
  Loss drops significantly

→ The model has learned TechNova's maternity leave policy.
```

Self-Supervised vs Supervised — The Difference

	Supervised FT	**Self-Supervised FT**
Data needed	500–1000 Q&A pairs	Your raw documents
Human effort	Medium (creating/generating pairs)	Very low (just feed documents)
What model learns	Exact Q→A patterns	Deep domain knowledge, terminology
Best used as	Step 2	Step 1 — always do this first

Best approach: Do self-supervised first, then supervised.

---

8. Fine-Tuning on the TechNova HR Document

The Setup

```
TechNova HR Policy Document:
  → 739 lines of HR policy text
  → ~25,000 words
  → Topics: Leave, Salary, ESOP, Attrition, L&D Budget

Step 1: Clean the text
  Remove special characters, fix encoding, remove extra whitespace.

Step 2: Create training examples (automatic — no labelling)
  Each 512-token window → predict next token.
  25,000 words → ~25,000 training examples, automatically.

Step 3: Format for fine-tuning
  Input:  [BOS] "Maternity leave at TechNova is 26 weeks"
  Target: "26 weeks, fully paid. Employees must have..."
           ↑ shifted by one token (next-token prediction)
```

Before and After

```
Before fine-tuning:
  Q: "What is ESOP?"
  A: "ESOP stands for Employee Stock Option Plan. It is a benefit
      that companies offer to employees allowing them to buy shares..."
  → Generic. Knows nothing about TechNova's ESOP.

After self-supervised fine-tuning on TechNova HR document:
  Q: "What is ESOP?"
  A: "At TechNova, ESOP vesting begins after 2 years of service.
      Employees at L3 and above are eligible. 25% of options vest
      each year over 4 years. The current grant price is determined
      at the time of joining."
  → Specific, accurate, TechNova-aligned.
```

9. PEFT — Parameter-Efficient Fine-Tuning

What Is PEFT and Why Does It Exist?

Full fine-tuning updates every single weight in the model. For a 7B parameter model that means updating 7,000,000,000 numbers every training step. This requires enormous GPU memory, takes days, and costs thousands of dollars.

PEFT is the family of techniques that asks: **can we fine-tune the model by updating only a tiny fraction of parameters, while still getting nearly the same quality as full fine-tuning?**

The answer, empirically, is yes. PEFT methods achieve 95–98% of full fine-tuning quality while training only 0.1%–1% of parameters.

PEFT Methods at a Glance

```
PEFT
  │
  ├── LoRA (Low-Rank Adaptation)
  │     How: Add small trainable matrices A and B alongside frozen weights
  │     Trains: ~0.1% of parameters
  │     Best for: Most fine-tuning tasks. Industry standard.
  │     Covered in: Section 10
  │
  ├── QLoRA (Quantised LoRA)
  │     How: LoRA + 4-bit quantisation of base model weights
  │     Trains: ~0.1% of parameters
  │     Memory: 8× less than LoRA
  │     Best for: Consumer GPUs, laptops, limited hardware
  │     Covered in: Section 11
  │
  ├── Prefix Tuning
  │     How: Add trainable "virtual tokens" prepended to the input
  │     Trains: Very few parameters (just the prefix embeddings)
  │     Best for: Tasks where the input format is fixed
  │     Not covered in detail here — LoRA is generally superior
  │
  └── Prompt Tuning
        How: Learn soft embeddings that steer the model's behaviour
```

```
Trains: Extremely few parameters (just embedding vectors)
Best for: Very constrained environments (e.g. API-only access)
Limitation: Quality is lower than LoRA for complex tasks
```

Why LoRA and QLoRA Are the Standard

Prefix Tuning and Prompt Tuning were early PEFT approaches. They work but have limitations — quality degrades on complex tasks and they are hard to tune. LoRA was introduced in 2021 by Microsoft Research and quickly became the industry standard because it delivers consistent quality across tasks, is easy to configure, and the adapters are tiny and portable. QLoRA extended LoRA in 2023 to run on consumer hardware by quantising the base model. Today, when people say PEFT, they usually mean LoRA or QLoRA in practice.

---

10. LoRA — How It Actually Works (With Full Calculations)

The Problem LoRA Solves

Full fine-tuning updates all billion parameters. For a 7B parameter model, that is 28 GB of gradients alone. LoRA solves this with one observation:

The changes made to model weights during fine-tuning have low intrinsic rank.

This means the meaningful adaptation can be captured by a much smaller matrix than the original. You do not need to update all 16 million numbers in a weight matrix. You only need to learn a small delta — the difference — which lives in a lower-dimensional space.

Simple analogy: A company has 10,000 employees and you want to teach them all a new process. You do not retrain every single person. You train 5 specialists — the LoRA adapters. All 10,000 employees consult those specialists when they need the new knowledge. The specialists are small, cheap to train, and the 10,000 employees do not change at all.

Where Does 4096 × 4096 Come From?

This is the question most guides never answer. Let's build up to it from scratch.

Step 1 — What Is a Transformer?

A transformer model processes text by converting each word/token into a vector of numbers. This vector is called an embedding. The size of this vector is called the **hidden dimension** or `d_model`.

For LLaMA 3 8B, the hidden dimension is **4096**. This means every single token (word piece) in your input is represented as a list of 4096 numbers inside the model.

Step 2 — What Is Attention?

The core mechanism of transformers is attention. Attention lets each token "look at" other tokens in the sequence to decide what information is relevant. To do this, each token's 4096-number vector is transformed into three different vectors: Query (Q), Key (K), and Value (V).

Each of these transformations is done by multiplying the input vector by a weight matrix.

Step 3 — The Weight Matrix Shape

The Query weight matrix (Wq) transforms an input of size 4096 into a Query vector of size 4096:

```
Input token vector:   shape (4096,)       — 4096 numbers
Query weight matrix:  shape (4096, 4096)  — the transformation matrix
Output Query vector:  shape (4096,)       — 4096 numbers

Multiplication: (4096,) × (4096, 4096) = (4096,)
```

Why 4096 × 4096? Because the input dimension is 4096 and the output dimension is also 4096. The matrix must be exactly that shape to perform this transformation. It is not a magic number — it comes directly from the model's design choice of hidden dimension = 4096.

The Full Picture for LLaMA 3 8B

```
Model design choices:
  Hidden dimension (d_model):     4096
  Number of attention heads:        32
  Number of transformer layers:     32
  Total parameters:            ~8 billion

Each attention layer has these weight matrices:
  Wq:  (4096 × 4096)  =  16,777,216 parameters  ← Query
  Wk:  (4096 × 4096)  =  16,777,216 parameters  ← Key
  Wv:  (4096 × 4096)  =  16,777,216 parameters  ← Value
  Wo:  (4096 × 4096)  =  16,777,216 parameters  ← Output

Just one attention layer = 4 × 16,777,216 = 67,108,864 parameters
```

```
32 layers × 67M = ~2.1 billion params in attention alone
(The rest come from feed-forward layers and embeddings)
```

So the answer to "where does 4096 × 4096 come from?" is: it comes from the model's hidden dimension. The designer chose 4096 as the size of the internal representation vector. Every matrix that transforms between the same space must be 4096 × 4096.

What LoRA Actually Does

LoRA freezes W entirely and adds two small matrices A and B alongside it. Only A and B are trained. W is never touched.

The effective weight used in the forward pass becomes:

```
W_effective = W  +  (B × A) × (alpha / r)
               ↑ frozen       ↑ adapter — the learned delta
```

Step-by-Step Calculation — LLaMA 3 8B, One Attention Layer

Original layer:

```
Original layer:   4096 × 4096  =  16,777,216 parameters
```

LoRA with r = 16:

```
Matrix A:   4096 × 16   =   65,536 parameters   ← trainable
Matrix B:     16 × 4096  =   65,536 parameters   ← trainable
             ──────────────────────────────────
Total new:                  131,072 parameters

Parameter reduction:
  131,072 ÷ 16,777,216  =  0.0078  =  0.78% of original

LoRA trains only 0.78% of the parameters that full fine-tuning
would need — for this one matrix.
```

Across all 32 layers, Wq + Wv:

```
Adapter pairs:       2 matrices (Wq + Wv) × 32 layers  =  64 adapters
Parameters per pair: 131,072
Total trainable:     64 × 131,072  =  8,388,608  (~8.4 million)

Total model params:  8,030,261,248  (~8 billion)
Trainable %:         8,388,608 / 8,030,261,248  =  0.104%

You train only 0.1% of model weights.
Empirically achieves 95–98% of full fine-tuning quality.
```

The Forward Pass — Concrete Numbers

```
Setup:
  x     = input vector, length 4096
  W     = frozen 4096 × 4096 weight matrix
  A     = trainable 4096 × 16 matrix
  B     = trainable 16 × 4096 matrix
  alpha = 32,  r = 16  →  scaling = alpha / r = 2.0

Step 1: base_output   = x × W           → shape (4096,)
Step 2: low_dim       = x × A           → shape (16,)    ← compressed to rank
Step 3: delta_output  = low_dim × B     → shape (4096,)  ← expanded back
Step 4: scaled_delta  = delta_output × 2.0
Step 5: final_output  = base_output + scaled_delta

The adapter compresses (4096 → 16) then expands (16 → 4096).
This bottleneck is what makes it parameter-efficient.
```

Initialisation — Why It Starts as Zero

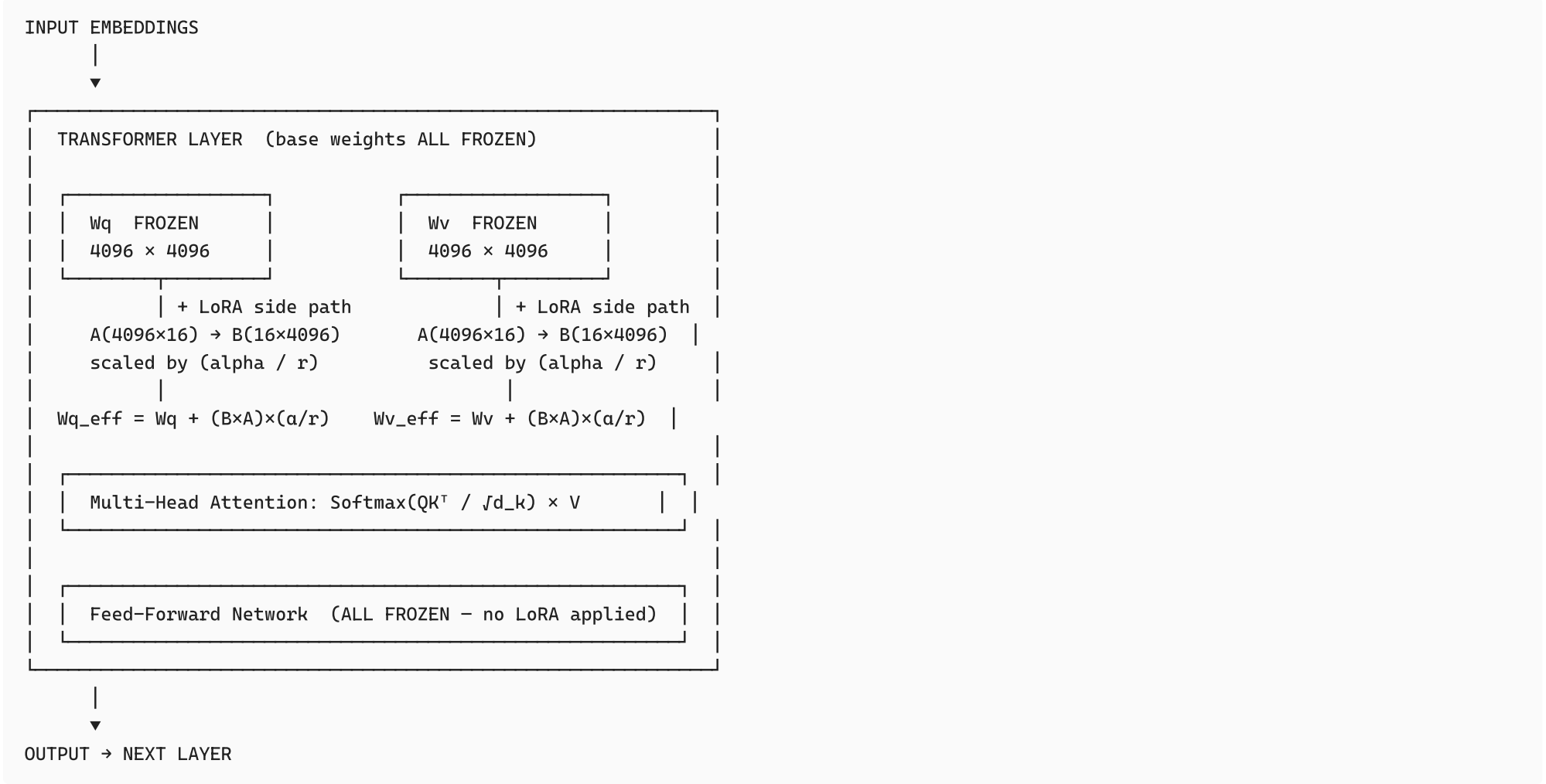
```
A ← random Gaussian  (small nonzero values)
B ← all zeros

At training step 0:
  B × A = 0  →  adapter contributes nothing
```

```
  Model output = x × W + 0  =  identical to base model

Training starts from a stable baseline.
Adapters gradually learn non-zero values as training progresses.
No sudden instability or destructive interference at the start.
```

LoRA Architecture — Inside a Transformer Layer



Alpha (α) — What It Is, Why It Exists, and a Full Example

What Is Alpha?

Alpha is a scaling hyperparameter that controls **how strongly the adapter influences the final output**. It is used in the formula:

```
output = x × W  +  (x × A × B) × (alpha / r)
                                  ─────────
                                  this ratio controls adapter strength
```

The adapter output is multiplied by `alpha / r` before being added to the base model output. If alpha is large relative to r, the adapter has more influence. If alpha is small relative to r, the adapter is more conservative.

Why Do We Need Alpha at All?

When you increase r, you are adding more adapter parameters. More parameters generally means the adapter can change the model's behaviour more dramatically. Alpha lets you separately control the *scale* of the change, independent of the rank. This decoupling makes tuning more predictable.

Think of it like this:

- **r** controls the *capacity* of the adapter — how complex a delta it can represent
- **alpha** controls the *volume* of the adapter — how loudly that delta speaks

Concrete Example

```
Setup: r = 16, alpha = 32

Adapter contribution = (x × A × B) × (alpha / r)
                     = (x × A × B) × (32 / 16)
                     = (x × A × B) × 2.0

Every number the adapter produces is multiplied by 2.0
before being added to the base model output.

Now compare with alpha = 8:
Adapter contribution = (x × A × B) × (8 / 16)
                     = (x × A × B) × 0.5

Now the adapter's contribution is halved.
The base model dominates more.
```

```
And with alpha = 16 (same as r):
Adapter contribution = (x × A × B) × (16 / 16)
                     = (x × A × B) × 1.0

The adapter is added with no scaling — neutral.
```

What Happens If Alpha Is Too High?

If alpha/r is very large (e.g. alpha=128, r=4 → ratio=32), the adapter dominates the base model output. The fine-tuned model may become unstable or lose general language understanding because the adapter is screaming over the base model's knowledge.

If alpha/r is very small (e.g. alpha=1, r=16 → ratio=0.0625), the adapter barely contributes anything. Fine-tuning will have little effect even after many epochs.

Alpha in Practice

```
Common settings:

  r=8,  alpha=8   → ratio = 1.0  (conservative, safe starting point)
  r=8,  alpha=16  → ratio = 2.0  (moderate — most commonly used)
  r=16, alpha=16  → ratio = 1.0  (conservative with more capacity)
  r=16, alpha=32  → ratio = 2.0  (moderate with more capacity)
  r=16, alpha=64  → ratio = 4.0  (strong adapter, use carefully)

Rule of thumb: start with alpha = 2 × r.
If the model is not adapting enough: increase alpha.
If the model is losing general ability: decrease alpha.
```

```python
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,   # ratio = 32/16 = 2.0 — moderate adaptation strength
    ...
)
```

The Rank Parameter r

r	Params added per matrix	Use Case
4	32,768 + 32,768 = 65,536	Simple style changes, very low memory
8	65,536 + 65,536 = 131,072	Standard — best balance of quality and cost
16	131,072 + 131,072 = 262,144	Complex domain adaptation, technical jargon
32	262,144 + 262,144 = 524,288	High quality, approaches full FT
64	524,288 + 524,288 = 1,048,576	Very close to full FT quality

Which Layers Get LoRA Adapters?

Layer	Apply LoRA?	Why
Wq — Query matrix	YES (default)	Controls what the model attends to — most impactful
Wv — Value matrix	YES (default)	Controls what info is extracted — second most impactful
Wk — Key matrix	Sometimes	Adds capacity for very different domains
Wo — Output projection	Sometimes	Marginal gains
Feed-forward layers	Aggressive setups	Use when domain is very different (code, science)
Token embeddings	Rarely	Only if new vocabulary tokens are needed

LoRA in Code

```
from peft import LoraConfig, get_peft_model, TaskType
from transformers import AutoModelForCausalLM

# Load base model
model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-3-8b")

# Configure LoRA
lora_config = LoraConfig(
    r=16,                          # rank — controls adapter size
    lora_alpha=32,                 # scaling: effective ratio = alpha/r = 2.0
    target_modules=["q_proj",      # apply to Query matrix (Wq)
                    "v_proj"],     # apply to Value matrix (Wv)
```

```
    lora_dropout=0.1,             # dropout to prevent overfitting
    bias="none",
    task_type=TaskType.CAUSAL_LM
)

# Inject adapters into model
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# Output:
# trainable params: 4,194,304 || all params: 8,030,261,248
# trainable%: 0.052%

# After training — merge into base (zero inference overhead)
merged = model.merge_and_unload()
merged.save_pretrained("./technova-hr-llama")
```

Does LoRA Need Labelled Data?

LoRA is a training method, not a data type. It is independent of your data choice.

- LoRA + Self-Supervised → no labels needed (just raw documents)
- LoRA + Supervised → needs (input, output) pairs
- LoRA + Instruction → needs (instruction, input, output) triples

Most practical setup: **QLoRA + Self-Supervised on your documents.** No labels, minimal GPU (~5 GB), under $5 on cloud, strong domain adaptation.

Deployment — Separate vs Merged

Option A — Keep adapter separate. Load base model then load adapter on top. Slight inference overhead. Useful when you have multiple adapters and want to hot-swap them on a single base model.

Option B — Merge adapter into base (most common). Compute `W_final = W + (B × A) × (alpha / r)` and save a single merged model. Zero overhead at inference.

```
merged = model.merge_and_unload()
merged.save_pretrained("./technova-hr-llama")
```

---

11. QLoRA — Fine-Tuning on a Laptop

The Problem LoRA Still Has

LoRA trains only small adapter matrices, but the full base model must still be loaded into GPU memory. A 7B model in float16 = 14 GB. Still too large for most consumer GPUs.

First — Understanding Bits and Number Formats

Before we can understand QLoRA, we need to understand what "32-bit", "16-bit", and "4-bit" actually mean. This is the foundation of everything in quantisation.

What Is a Bit?

A bit is the smallest unit of computer storage. It holds either 0 or 1.

How Many Numbers Can You Represent With N Bits?

With N bits, you can represent 2^N different values:

```
1 bit  → 2 values         (0 or 1)
4 bits → 16 values        (0 through 15)
8 bits → 256 values       (0 through 255)
16 bits → 65,536 values
32 bits → 4,294,967,296 values  (about 4 billion)
```

More bits = more values = more precision = more memory.

How a Neural Network Weight Is Stored

Every single weight (parameter) in a neural network is a decimal number like 0.3847 or -1.2341. The computer needs to store this number in binary. The format used determines both memory usage and precision.

float32 (32-bit floating point) — the standard:

```
32 bits = 4 bytes per weight

Structure of 32 bits:
  1 bit  → sign (positive or negative)
```

```
  8 bits → exponent (how large the number is)
  23 bits → mantissa (the actual digits of precision)

Range:  approximately −3.4×10³⁸ to +3.4×10³⁸
Precision: about 7 decimal digits

Example: 0.3847 stored as float32
  Sign:     0           (positive)
  Exponent: 01111101    (encodes the scale)
  Mantissa: 10001100100001110010101  (encodes the digits 0.3847)
  Total:    32 bits = 4 bytes
```

float16 (16-bit floating point) — half precision:

```
16 bits = 2 bytes per weight  (2× smaller than float32)

Structure of 16 bits:
  1 bit  → sign
  5 bits → exponent
  10 bits → mantissa

Range:  approximately −65,504 to +65,504
Precision: about 3 decimal digits

Memory saving vs float32:   2× less memory
Quality loss:               small, usually acceptable for inference
```

int8 (8-bit integer):

```
8 bits = 1 byte per weight  (4× smaller than float32)

Can represent 256 integer values: −128 to +127
No decimal precision — only whole numbers

Memory saving vs float32:   4× less memory
Quality loss:               noticeable for some tasks
```

4-bit NF4 (Normal Float 4) — used in QLoRA:

```
4 bits = 0.5 bytes per weight  (8× smaller than float32)

Can represent only 16 values
But these 16 values are chosen cleverly (see below)

Memory saving vs float32:   8× less memory
Quality loss:               surprisingly small (QLoRA's key insight)
```

Memory Impact — Why Bits Matter So Much

```
One weight stored in different formats:
  float32:   4 bytes
  float16:   2 bytes
  int8:      1 byte
  4-bit NF4: 0.5 bytes

Now multiply by 7 billion weights (7B model):
  float32:   7B × 4 bytes   = 28,000 MB  = 28 GB
  float16:   7B × 2 bytes   = 14,000 MB  = 14 GB
  int8:      7B × 1 byte    =  7,000 MB  =  7 GB
  4-bit NF4: 7B × 0.5 bytes =  3,500 MB  =  3.5 GB

This is why bit format matters — it directly controls how much GPU
memory you need just to load the model before any training begins.
```

QLoRA's Solution — Quantisation + LoRA

QLoRA combines two things: it loads the base model in 4-bit (to save memory) and adds LoRA adapters in float32 (to train with precision).

```
QLoRA idea:
  Base model weights:  store in 4-bit  → only 3.5 GB for a 7B model
  LoRA adapters:       keep in float32  → ~33 MB, trained normally
  During forward pass: temporarily expand 4-bit → float16 for computation
  Gradients:           only flow through adapters (not base weights)
```

```
Result: you get the memory savings of 4-bit
        AND the training quality of full-precision adapters
```

What Quantisation Actually Does — Step by Step

Quantisation means converting a high-precision number (like float32) into a low-precision representation (like 4-bit) that takes less memory. The key question is: which 16 values do you choose to represent with your 4 bits?

Naive 4-bit would just evenly space 16 values across a range. But NF4 is smarter.

Why NF4 Is Better Than Naive 4-bit

Neural network weights follow a **normal (Gaussian) distribution** — they cluster around zero. Most weights are small numbers near 0. Very few weights are large numbers far from 0.

If you space your 16 values evenly from -1.0 to +1.0, you waste most of your 16 slots on the rare large values and have too few slots for the common small values near zero.

NF4 spaces its 16 values based on the **quantiles of the normal distribution** — more slots near zero where weights actually cluster, fewer slots at the extremes. This dramatically reduces the error when approximating real weights.

The Full NF4 Calculation — A Worked Example

```
The 16 NF4 values (fixed, based on normal distribution quantiles):

  Index:  0      1       2       3       4       5       6       7
  Value: -1.0  -0.6962 -0.5251 -0.3949 -0.2844 -0.1848 -0.0911  0.0

  Index:  8      9      10      11      12      13      14
  Value: 0.0911 0.1848  0.2844  0.3949  0.5251  0.6962  1.0

Notice: values are denser near 0, sparser near ±1.0.
This matches where neural network weights actually live.
```

Now let's quantise a real weight value step by step:

Weight value to quantise: 0.3847 (stored originally as float32, 4 bytes)

```
Step 1 — Normalise the weight to the range [−1, +1]
  (Weights are normalised per block of 64 values using their max absolute value)
  Assume max absolute value in this block = 0.95
  Normalised value = 0.3847 / 0.95 = 0.4050

Step 2 — Find the nearest NF4 value
  NF4 values near 0.4050:
    Index 11 → 0.3949  (distance = |0.4050 − 0.3949| = 0.0101)
    Index 12 → 0.5251  (distance = |0.4050 − 0.5251| = 0.1201)
  Nearest: index 11, value 0.3949

Step 3 — Store the index, not the value
  Store index 11 in binary: 1011
  That is 4 bits instead of 32 bits → 8× smaller!

  Memory used: 4 bits = 0.5 bytes  (was 4 bytes in float32)

Step 4 — At compute time: dequantise
  Read 4-bit index 1011 → 11
  Look up NF4 table → 0.3949
  Multiply by stored scale factor 0.95 → 0.3952

  Original value:  0.3847
  Recovered value: 0.3952
  Error:           |0.3952 − 0.3847| = 0.0105  (very small)
```

The key insight: **we are not storing the exact number, we are storing which of the 16 buckets it falls into.** The error is small because NF4's buckets are cleverly positioned where weights actually are.

Another Example with a Different Weight

```
Weight value: −0.1523

Step 1 — Normalise: −0.1523 / 0.95 = −0.1603

Step 2 — Find nearest NF4 value:
  Index 5 → −0.1848  (distance = |−0.1603 − (−0.1848)| = 0.0245)
  Index 6 → −0.0911  (distance = |−0.1603 − (−0.0911)| = 0.0692)
```

```
  Nearest: index 5, value -0.1848

Step 3 — Store index 5 = binary 0101 → 4 bits

Step 4 — Dequantise: -0.1848 × 0.95 = -0.1756
  Original: -0.1523
  Recovered: -0.1756
  Error: 0.0233 — slightly larger but still small
```

Memory Calculation — Side by Side

```
Model: LLaMA 3 8B

Full Fine-Tuning (float32):
  Weights:    8B × 4 bytes =  32 GB
  Gradients:  8B × 4 bytes =  32 GB
  Optimizer:  8B × 8 bytes =  64 GB  (Adam: 2 moments × 4 bytes)
  TOTAL:                    128 GB   → Needs 2× A100 (80 GB each)

LoRA (float16 base + float32 adapters):
  Base weights:   8B × 2 bytes =  16 GB
  Adapter + grads + optimizer: ~650 MB
  TOTAL:                       ~17 GB → Needs 1× A100

QLoRA (4-bit NF4 base + float32 adapters):
  Base weights:  8B × 0.5 bytes =  4.0 GB
  Adapter:       8.4M × 4 bytes =   33 MB
  Gradients:     adapters only  =   33 MB
  Optimizer:     adapters only  =   67 MB
  KV Cache (estimate):          ~500 MB
  TOTAL:                        ~4.6 GB → Fits on RTX 3090!
```

What Happens During QLoRA Training Step by Step

```
1. Load base model in 4-bit NF4 (frozen, no gradients)
   → 4 GB on GPU
```

```
2. Add LoRA adapters in float32 (trainable)
   → 33 MB additional

3. Forward pass:
   - For each layer: dequantise 4-bit weights → float16 on the fly
   - Compute attention using the temporary float16 weights
   - Add LoRA adapter contribution (float32 → float16 for computation)
   - After layer is done: discard the float16, keep the 4-bit

4. Backward pass:
   - Compute gradients ONLY for LoRA adapters
   - Base model weights have no gradients (frozen)
   - Optimizer updates only the adapter parameters

5. Memory during training:
   - 4-bit base: always 4 GB
   - One layer's float16 expansion: temporary, ~600 MB, released after each layer
   - Adapter gradients and optimizer states: ~100 MB
   - Peak memory: ~5-6 GB
```

QLoRA in Code

```python
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
import torch

# Step 1: Configure 4-bit quantisation
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",              # use Normal Float 4 format
    bnb_4bit_compute_dtype=torch.float16,  # expand to float16 for computation
    bnb_4bit_use_double_quant=True          # also quantise the scale factors
)

# Step 2: Load base model in 4-bit
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3-8b",
    quantization_config=bnb_config,
```

```
    device_map="auto"
)

# Step 3: Prepare for k-bit training
# (adds gradient checkpointing, converts some layers for compatibility)
model = prepare_model_for_kbit_training(model)

# Step 4: Add LoRA adapters — exactly the same as pure LoRA
lora_config = LoraConfig(
    r=16,
    lora_alpha=32,                          # ratio = 2.0
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.1,
    task_type="CAUSAL_LM"
)
model = get_peft_model(model, lora_config)

# Step 5: Train
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="./technova-hr-model",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    learning_rate=2e-4,
    fp16=True,
    logging_steps=10,
    save_steps=100,
)
# trainer.train()
# Only the 8.4M adapter parameters receive gradients.
# 4-bit base weights: frozen, no gradients, no optimizer state.
```

---

12. Training Parameters — What They Mean and How to Set Them

This is the section most tutorials skip. These numbers look arbitrary but each one has a specific meaning and a real impact on your results.

num_train_epochs

How many times the model sees your entire training dataset from start to finish. One pass through all your training examples = one epoch.

Too few epochs → underfitting. Too many epochs → overfitting (model memorises training data and fails on new questions).

```
Typical values:
  Self-supervised on documents:     1–3 epochs
  Supervised Q&A fine-tuning:       3–5 epochs
  Instruction fine-tuning:          3–5 epochs
  Very small dataset (<500 pairs):  5–10 epochs

Watch validation loss. If it starts going UP while training loss goes DOWN
→ overfitting. Stop at the point where validation loss is lowest.
```

learning_rate

How large a step the model takes to adjust its weights after each batch. Too large and it overshoots. Too small and it learns painfully slowly.

```
learning rate too high (e.g. 1e-2) → unstable training, loss diverges
learning rate too low  (e.g. 1e-6) → training barely moves
learning rate just right           → loss smoothly decreases

Typical values for fine-tuning:
  LoRA / QLoRA:             2e-4  (0.0002)  ← most common starting point
  Full fine-tuning:         1e-5  (0.00001) ← much smaller, careful steps
  Instruction fine-tuning:  1e-4 to 2e-4
  Very small dataset:       1e-4

The notation:
  2e-4  means  2 × 10⁻⁴  =  0.0002
  1e-5  means  1 × 10⁻⁵  =  0.00001
```

per_device_train_batch_size

How many training examples the model looks at together before updating its weights once. Think of it like studying — you can read 1 question at a time and update your notes after each, or read 4 questions together and update once. Reading 4 together gives a better, more stable update.

```
batch_size = 1  → model updates weights after every single example
                  Very noisy updates. Unstable training.

batch_size = 4  → model looks at 4 examples together, computes average
                  gradient, then updates once. More stable.

batch_size = 32 → very smooth gradients, very stable, but needs much
                  more GPU memory to hold 32 examples at once.

Rule: bigger batch = more stable training = more GPU memory needed.

Typical values:
  Consumer GPU (RTX 3090, 24 GB):   2–4
  A100 (80 GB):                     8–32

If you get OOM (out of memory) error:
  Reduce batch_size and increase gradient_accumulation_steps.
  batch_size=2, gradient_accumulation_steps=4 → equivalent to batch_size=8
  but uses 4× less memory.
```

What Is a Step?

A step is one weight update. Every time the model processes one batch and updates its weights — that is one step.

```
Formula:
  Total steps = (number of training examples ÷ batch_size) × num_epochs

Example:
  Training examples : 80
  batch_size        : 4
  num_epochs        : 8

  Steps per epoch   = 80 ÷ 4  = 20 steps
  Total steps       = 20 × 8  = 160 steps

  At each step:
    Step 1  → model sees examples 1, 2, 3, 4    → updates weights
    Step 2  → model sees examples 5, 6, 7, 8    → updates weights
```

```
    Step 3  → model sees examples 9, 10, 11, 12 → updates weights
    ...
    Step 20 → end of epoch 1
    Step 21 → epoch 2 begins, sees examples 1, 2, 3, 4 again
    ...
    Step 160 → training complete

When logging_steps = 5:
  Every 5 steps the loss is printed to screen.

  Step  5 | Loss: 2.84  ← high loss, model knows nothing yet
  Step 10 | Loss: 2.31  ← dropping, model starting to learn
  Step 15 | Loss: 1.89
  Step 20 | Loss: 1.52
  ...
  Step 160 | Loss: 0.43 ← low loss, model has learned the data

  Loss going DOWN = model is learning              ✅
  Loss going UP   = overfitting, stop training     ⚠️
  Loss not moving = learning rate too low          ⚠️
  Loss jumping around = learning rate too high     ⚠️
```

In your TrainingArguments these three parameters work together:

```
training_args = TrainingArguments(
    per_device_train_batch_size = 4,   # 4 examples per step
    num_train_epochs            = 8,   # 8 passes over full dataset
    logging_steps               = 5,   # print loss every 5 steps
    # Total steps = (80 ÷ 4) × 8 = 160 steps total
)
```

gradient_accumulation_steps

Instead of updating weights every batch, accumulate gradients over N batches then update. Simulates a larger batch size without the memory cost.

```
# Equivalent to batch_size=16 but uses 4× less memory:
per_device_train_batch_size=4,
```

```
gradient_accumulation_steps=4,
```

warmup_steps / warmup_ratio

At the very start of training, use a smaller learning rate and gradually increase it. This prevents the model from making large destructive updates before it has seen enough data.

```
warmup_ratio=0.03  → warm up for the first 3% of training steps
warmup_steps=100   → warm up for the first 100 steps exactly
Typical: warmup_ratio=0.03
```

weight_decay

Slightly penalises large weight values, preventing the model from becoming too confident about specific training examples. Reduces overfitting.

```
Typical value: 0.01
```

lora_dropout

During training, randomly zero out some adapter activations to prevent the adapter from memorising training examples. Dropout is only active during training.

```
Typical value: 0.05 to 0.1
  0.0  → no dropout (risk of overfitting)
  0.1  → 10% of adapter activations zeroed during training
```

Complete Training Configuration — TechNova HR Example

```
from transformers import TrainingArguments

training_args = TrainingArguments(
    output_dir="./technova-hr-model",

    # How long to train
    num_train_epochs=3,

    # Batch size and memory
```

```
    per_device_train_batch_size=4,
    gradient_accumulation_steps=2,        # effective batch = 4 × 2 = 8

    # Learning rate
    learning_rate=2e-4,
    warmup_ratio=0.03,
    lr_scheduler_type="cosine",           # cosine decay after warmup

    # Regularisation
    weight_decay=0.01,

    # Precision
    fp16=True,

    # Logging and saving
    logging_steps=10,
    save_steps=100,
    save_total_limit=2,
    evaluation_strategy="steps",
    eval_steps=100,
    load_best_model_at_end=True,
)
```

How to Read Training Output

```
Step   10 | Loss: 2.84 | LR: 0.000040  ← warm-up, LR low, loss high
Step   50 | Loss: 1.92 | LR: 0.000180  ← LR climbing, loss dropping fast
Step  100 | Loss: 1.41 | LR: 0.000200  ← LR at peak, loss still falling
Step  200 | Loss: 1.18 | LR: 0.000180  ← LR decaying, loss stabilising
Step  500 | Loss: 0.97 | LR: 0.000100  ← good zone
Step 1000 | Loss: 0.89 | LR: 0.000050  ← converging

Loss stops decreasing  → training is done, more epochs won't help
Loss starts increasing → overfitting, stop training now
Loss is unstable/spiky → learning rate too high, reduce it
```

---

13. What Gets Updated — A Layer-by-Layer View

```
Transformer Layer (one of 32 in LLaMA 3 8B):

  ┌──────────────────────────────────────────────┐
  │ Layer Norm                  → FROZEN         │
  │                                              │
  │ Multi-Head Attention:                        │
  │   Wq (Query matrix)         → FROZEN + LoRA A,B │
  │   Wk (Key matrix)           → FROZEN         │
  │   Wv (Value matrix)         → FROZEN + LoRA A,B │
  │   Wo (Output projection)    → FROZEN         │
  │                                              │
  │ Layer Norm                  → FROZEN         │
  │                                              │
  │ Feed-Forward Network:                        │
  │   W_up (up projection)      → FROZEN         │
  │   W_gate (gate projection)  → FROZEN         │
  │   W_down (down projection)  → FROZEN         │
  └──────────────────────────────────────────────┘

Full fine-tuning: ALL weights updated.
LoRA (typical):   Only Wq and Wv get adapter matrices.
LoRA (aggressive): All attention matrices + FF layers get adapters.

Base model knowledge → preserved in frozen weights.
New domain knowledge → stored in small adapter matrices.
```

After Training — Merging Adapters

```
Option A: Keep adapter separate
  Load base model + load adapter.
  Slight overhead at inference.
  Useful when you want to hot-swap multiple adapters.

Option B: Merge adapter into base (most common for deployment)
  W_final = W + (B × A) × (alpha / r)
  Zero overhead at inference.
```

```
  Cannot separate base from adapter after merging.

model = model.merge_and_unload()
model.save_pretrained("./technova-hr-llama")
```

---

14. Common Problems and Fixes

Catastrophic Forgetting

The model forgets how to do general tasks after fine-tuning on domain data.

- Use LoRA: frozen base weights preserve general knowledge. Adapters only add new knowledge on top.
- Mix data: include 10–20% general text with 80–90% domain text in training.
- Lower learning rate: use 1e-5 instead of 1e-3. Smaller steps = less forgetting.

Overfitting to Training Data

The model memorises the HR document word-for-word. Training loss → 0 but test loss stays high.

- Add dropout: `lora_dropout=0.1` in LoRA config
- Train for fewer epochs: 3, not 10
- Add more diverse training examples
- Use weight decay: `weight_decay=0.01`

Mode Collapse

The model produces the same answer for every question.

Cause: training data had too little variety in responses. Fix by diversifying examples — different topics, different question types, different answer lengths.

How to Know Fine-Tuning Is Working

```
Compare on held-out questions (not in training data):

Before fine-tuning:
```

```
  Q: "What is the ESOP vesting schedule?"
  A: "ESOP vesting schedules typically span 4 years with a 1-year cliff..."
     (generic, no TechNova specifics)

After fine-tuning:
  Q: "What is the ESOP vesting schedule?"
  A: "At TechNova, ESOP options vest over 4 years: 25% after year 1,
      then 25% each year after. Minimum L3 grade required for eligibility."
     (specific, accurate, TechNova-aligned)

Metrics to track:
  Perplexity on domain text     → lower is better
  Human evaluation on Q&A       → compare before/after answers
  ROUGE / BERTScore on test set → quantitative comparison
```

---

15. Summary — When to Do What

Situation	Best Approach
Have documents, no labels, quick start	Self-supervised FT on raw documents
Want specific Q&A behaviour	Supervised FT with 500+ pairs (auto-generate ~$2)
Need specific tone / format / role	Instruction FT with 100+ examples
Consumer GPU < 8 GB	QLoRA, r=8
Consumer GPU 8–24 GB	QLoRA, r=16
Professional GPU 24–80 GB	LoRA, r=16 or r=32
Multiple A100s	Full fine-tuning (rare, maximum quality)
Facts change frequently	RAG, not fine-tuning
Need source citations	RAG, not fine-tuning
Production-grade system	Fine-tune for domain + RAG for current facts

The LoRA Decision Tree

```
GPU memory?
  < 8 GB   → QLoRA, r=8
  8–24 GB  → QLoRA, r=16
  24–80 GB → LoRA, r=16 or r=32
  > 80 GB  → Full fine-tuning (rarely needed)

How different is your domain from general text?
  Slightly different (new terminology)  → r=8,  alpha=16
  Quite different (technical jargon)    → r=16, alpha=32
  Very different (code, formulas, etc.) → r=64, alpha=64

How much data do you have?
  < 1,000 examples → r=4 or r=8,   1–3 epochs
  1,000–10,000     → r=16,          3–5 epochs
  > 10,000         → r=32+,         5–10 epochs

Do you have labelled data?
  No  → Self-supervised FT (just feed raw documents)
  Yes → Supervised FT, or do self-supervised first then supervised
```

Key Numbers to Remember

```
LoRA r=16 on 7B model:
  Trainable params:          8,388,608  (0.1% of total)
  Reduction per matrix:      131,072 ÷ 16,777,216 = 0.78% of original
  Memory for adapters:       ~33 MB
  Training time on A100:     hours not days

QLoRA 4-bit on 7B model:
  Total GPU memory needed:   ~4.6 GB   (vs 128 GB full fine-tuning)
  Memory saving:             8× reduction from float32 → 4-bit
  Quality vs full FT:        95–98%
  Runs on:                   consumer RTX 3090 (24 GB)

Self-supervised on 25,000-word document:
  Training examples:         ~25,000  (created automatically)
  Training time:             30–60 minutes on one GPU
  Cloud cost:                under $5
```

```
Auto-generating SFT labels with GPT:
  100 chunks × 5 pairs:      500 training examples
  Cost:                      ~$2 in API fees
  Time:                      5–10 minutes
```
